THE OPEN SOURCE AUDIT

A Capstone Project — Open Source Software (NGMC)

Chosen Software: Git

The distributed version control system created by Linus Torvalds

Student Name	[Abhay Raj]
Registration No.	[24BAI10391]
Slot	[E22]
Date of Submission	[31-03-2026]
Course	Open Source Software | Units 1–5
Max Marks	100 (Report: 60 | Scripts: 40)

Part A — Origin and Philosophy

This section covers Units 1 and 2 of the course. It documents the origin story of Git, a thorough analysis of its license, and a personal ethical reflection on what open-source software means for society and the profession of software development.

A1. The Problem Git Was Created to Solve

The Context: Linux and the Collaboration Crisis of 2005

In the early 2000s, the Linux kernel was one of the most ambitious collaborative software projects in human history. Thousands of developers across the world were contributing patches, fixes, and features to a single codebase — the very foundation of operating systems running servers, supercomputers, and embedded devices. Managing all of this required a version control system (VCS): a tool to track changes, allow branching for experimentation, and merge contributions from many hands without losing history or introducing chaos.

For years, the Linux kernel project used BitKeeper — a proprietary version control system developed by BitMover Inc. BitKeeper was technically capable, and BitMover allowed the Linux community to use it at no cost, as long as they did not develop a competing product. This arrangement was always uneasy. Richard Stallman and other Free Software advocates objected on philosophical grounds: the most important open-source project in the world was dependent on a tool whose source code was secret and whose usage rules could change at any moment.

The crisis arrived in April 2005. BitMover withdrew the free-use license for the Linux kernel project, citing concerns that a Linux developer, Andrew Tridgell, had begun reverse-engineering the BitKeeper protocol. Whatever the exact circumstances of the dispute, the outcome was sudden and severe: the Linux kernel project had no version control system it was allowed to use.

Linus Torvalds' Response: Build a Better Tool

Faced with this situation, Linus Torvalds — the creator of Linux — did not look for another proprietary replacement or accept the terms of an alternative commercial tool. He sat down and, over the Easter weekend of April 2005, wrote the first version of Git from scratch.

His requirements were specific and driven by frustration with every existing solution he had evaluated. Torvalds wanted a system that was fast — capable of handling the scale of the Linux kernel (tens of thousands of files, millions of lines of code). He wanted it to be distributed: every developer would have a full copy of the entire history, not just a working snapshot. This meant that work could happen offline, merges could be previewed locally, and no single server was a single point of failure. He also demanded

cryptographic integrity — Git uses SHA-1 hashing to ensure that no part of a repository's history can be altered without detection.

The result was a system unlike anything else available at the time. Git was not designed to be easy for beginners. It was designed to be technically correct, blazingly fast, and structurally sound. Torvalds later described the approach plainly: he wanted 'the anti-CVS'. CVS, the dominant open-source VCS of the era, was notoriously fragile and slow. Git was designed from day one as its philosophical and technical opposite.

Why It Had to Be Open Source

Torvalds released Git under the GNU General Public License version 2 (GPL v2) from the very beginning. This was not incidental — it was the point. The crisis that created Git was caused by dependency on proprietary software. Torvalds understood that if Git were to serve the Linux community, it had to be something the community could trust, audit, modify, and control. A proprietary Git would have been a contradiction in terms: a locked tool for a free project.

Within a few weeks of its creation, Git was stable enough that the Linux kernel project migrated its entire history into it. Within a year, it was being adopted by projects far beyond the Linux world. By 2008, GitHub was founded, building a platform entirely on top of Git's open protocols. Today, Git is the dominant version control system across the entire software industry. It is estimated that over 90% of software developers use Git as their primary VCS.

The story of Git is, at its core, a story about what happens when a proprietary tool fails a community that depends on it, and that community chooses to build something free rather than accept the same vulnerability twice.

A2. The License — What It Actually Says

Git's License: GNU General Public License v2

Git is released under the GNU General Public License version 2, known as the GPL v2. This is one of the oldest and most widely used open-source licenses in the world, written by Richard Stallman and the Free Software Foundation. The full text of the license is available at https://www.gnu.org/licenses/old-licenses/gpl-2.0.html and is included in Git's source repository as the file 'COPYING'.

The Four Freedoms of Free Software

The GPL v2 is designed to guarantee what the Free Software Foundation calls the Four Freedoms. These are not merely permissions — they are rights that the license actively protects and extends to every user:

- Freedom 0: The freedom to run the program, for any purpose. You may use Git to manage your personal poetry, your startup's codebase, or a government system's configuration. The license imposes no restrictions on use.

- Freedom 1: The freedom to study how the program works, and to change it so it does your computing as you wish. Access to the source code is the prerequisite for this freedom. You may download Git's source, read every line, and modify it for your needs.
- Freedom 2: The freedom to redistribute copies so you can help others. You may give Git to a friend, package it in your own Linux distribution, or include it in your company's internal tooling.
- Freedom 3: The freedom to distribute copies of your modified versions to others. If you improve Git, you may share your improved version with the world — and the GPL v2 says you must do so under the same license.

Git's GPL v2 license grants all four freedoms. This is what makes Git genuinely free software, as opposed to merely free-of-charge software.

Can a Company Sell a Modified Git Without Sharing Changes?

No. This is the most important practical consequence of the GPL v2 — the copyleft clause. The GPL v2 is specifically designed to prevent this scenario. If you take Git's source code, modify it, and distribute that modified version to others (including selling it), the GPL v2 requires you to provide the complete source code of your modified version under the same GPL v2 terms.

This is often called a 'share-alike' or 'viral' provision. It ensures that improvements to GPL-licensed software cannot be absorbed into proprietary products and locked away. The business model of companies like Red Hat (now IBM) works around this by selling support, certification, and services around open-source software — not by locking away the code itself.

It is worth noting that this restriction applies to distribution. If a company modifies Git for internal use only and does not distribute it, they are not required by the GPL to release those changes. But the moment they ship a product containing modified Git source code to a customer or user outside their organization, the copyleft obligation triggers.

GPL vs MIT: A Practical Comparison

The difference between the GPL and the MIT license is fundamentally a difference in philosophy about what freedom means.

The MIT license is a permissive license. It allows anyone to take the source code, use it, modify it, and redistribute it — including as part of a closed, proprietary product — with almost no conditions. The only requirement is that the original copyright notice is preserved. This is the 'free beer' model of open source: use it however you like.

The GPL is a copyleft license. It extends freedom but also enforces it. Anyone who distributes a GPL-licensed program must pass along the same freedoms to the people they distribute it to. This is the 'free as in freedom' model: not just free to use, but free to stay free.

For my own projects, I would choose based on context. For a utility library meant to be embedded in other people's work — including commercial work — I would use MIT, because broad adoption matters more than enforcing openness. For a standalone application or a tool intended to build a community around it, I would use GPL v2, because I would want any improvements to remain in the commons. Git

itself made the right choice: the GPL v2 is why Git cannot be forked into a proprietary product and then have its own community locked out of improvements.

Free as in Beer vs Free as in Freedom

The phrase 'free as in free beer' describes software that costs nothing to obtain. The phrase 'free as in freedom' (libre) describes software that grants you rights over how you use, study, modify, and share it. These two meanings of 'free' are entirely independent. Software can be free-of-charge but not free (a company gives you a free binary but keeps the source secret). Software can be free-as-in-freedom but not free-of-charge (Red Hat Enterprise Linux is GPL-licensed but Red Hat charges for subscriptions).

Git is both. You can download and use it at no cost. You can also read every line of its source code, modify it, and redistribute your changes. The GPL v2 ensures that the second kind of freedom cannot be taken away from you by any company that chooses to build on Git's foundation.

A3. The Ethics of Open Source

Should All Software Be Open Source?

This question deserves a genuine two-sided answer, not a reflexive yes from enthusiasm or reflexive no from commercial instinct.

The argument for universal open source is compelling. Software increasingly governs critical infrastructure: voting systems, hospital equipment, financial systems, traffic management, aircraft control. The public has a legitimate interest in being able to audit and verify systems that affect their lives. Proprietary software in these contexts creates an accountability gap: a company can claim their system is secure, fair, or correct, and there is no independent way to verify that claim. Furthermore, there is a strong argument from efficiency: when public institutions build proprietary software with public funds, the resulting code is locked away, preventing other institutions from building on it. Open source maximizes the return on public investment.

The argument against is also real. Software development requires sustained investment in time, talent, and infrastructure. Many individuals and companies justify that investment by charging for proprietary access to the software they build. If every piece of software were required to be open source, some categories of software investment — particularly niche enterprise tools — might not happen at all. Secrecy also has genuine security benefits in specific contexts: a system whose internal workings are not published is harder for adversaries to probe and exploit. Security through obscurity is not a sufficient defense, but it is a real layer.

My view is that universal mandatory open source is not the right answer, but that the current balance is tilted too far toward proprietary. Critical public infrastructure, software built with public funding, and systems whose failures affect large numbers of people should carry a strong presumption of open

access. Commercial software that differentiates genuinely innovative work should be allowed to be proprietary, with the market and competition as the check on abuse.

Is It Ethical to Profit from Free Community Labor?

This is the most uncomfortable question in the open-source ecosystem, and it is one that companies like Google, Meta, and Amazon face directly. These companies use Linux, contribute to some projects, and derive enormous commercial value from the open-source commons — value that far exceeds any contributions they make back.

The strict legal answer is that this is permitted. The GPL and MIT licenses both allow commercial use. The open-source community made a deliberate choice to use these licenses, and those licenses do not restrict commercial exploitation.

The ethical answer is more complicated. There is a difference between a company that takes from the commons and also gives back — contributing bug fixes, funding maintainers, open-sourcing their own tools — and a company that extracts value without any reciprocity. The latter is legal but ethically questionable, and it has real consequences: open-source maintainers regularly burn out from the pressure of maintaining widely-used software as an unpaid side project while corporations derive billions of dollars of value from their work.

The Log4Shell vulnerability of 2021 made this tension visible. A critical security flaw in the open-source library Log4j — maintained by a small group of volunteers — affected virtually every major company's infrastructure. The maintainers received no significant financial support before the crisis and were overwhelmed by demands to fix it immediately. This is the structural problem: the commons is real, the extraction is real, but the investment in maintaining the commons is wildly insufficient.

Standing on the Shoulders of Giants

The phrase comes from Isaac Newton, who wrote that if he had seen further than others, it was by standing on the shoulders of giants. In software, this metaphor is lived reality. Every piece of software built today — from a student's first web application to a space telescope's control system — runs on layers of open-source foundations: the Linux kernel, the GNU toolchain, OpenSSL, SQLite, Python, Git.

Open source enables this kind of cumulative progress in a way proprietary ecosystems cannot. When knowledge is locked away, every generation must partially re-discover what the previous generation already knew. When knowledge is shared, each generation builds from the highest point the previous generation reached.

The question of whether open source undermines innovation is often framed as a concern that giving code away removes the incentive to innovate. The evidence suggests the opposite: open source accelerates innovation by reducing duplication, enabling collaboration at scales no single company can match, and creating ecosystems of complementary tools that collectively advance the state of the art faster than any proprietary alternative could.

Git itself is the clearest example of this. Its open design has made it the substrate for an entirely new generation of development practices: continuous integration, pull-request-based code review, infrastructure-as-code, and open collaboration at a global scale. None of this would have been possible if Torvalds had kept Git proprietary.

Part B — Linux Footprint

This section documents how Git is installed, managed, and organized on a Linux system. All commands were tested on Ubuntu 22.04 LTS. Results will vary slightly on RHEL/Fedora systems.

Installation Method

Git is available in the package repositories of virtually every major Linux distribution. On Debian/Ubuntu-based systems it is installed via the APT package manager:

```
sudo apt update
sudo apt install git
git --version
# Output: git version 2.43.0
```

On RHEL/Fedora/CentOS systems, the equivalent commands use DNF:

```
sudo dnf install git
git --version
```

Git can also be compiled from source — useful when a newer version is needed than the distribution packages. The source is available at https://github.com/git/git. Compilation requires build tools (gcc, make) and library dependencies (libcurl, libexpat, openssl-dev).

Key Directories

Git's files are distributed across several standard Linux filesystem locations, following the Filesystem Hierarchy Standard (FHS):

- /usr/bin/git — The main Git executable binary. This is what runs when you type git at the command line.
- /usr/lib/git-core/ — Git's helper programs and sub-commands (git-fetch, git-merge, git-diff, etc.). Each of these is a separate binary called by the main git command.
- /usr/share/doc/git/ — Documentation, changelogs, and the license file (COPYING).
- /usr/share/man/man1/ — Man page files for every Git command. Accessible via man git-commit, man git-push, etc.
- /etc/gitconfig — System-wide Git configuration. Settings here apply to all users on the system.
- ~/.gitconfig — Per-user global Git configuration. Created when you run git config --global. Stores your name, email, preferred editor, etc.
- .git/ inside each repository — The hidden directory that is a Git repository. Contains the object database, refs, HEAD pointer, config file, and hooks. This is where all version history lives.

User and Group

The main git binary (/usr/bin/git) is owned by root and belongs to the root group, with permissions set to 755 (rwxr-xr-x). This means:

- root owns the binary — only root or the package manager can update or replace it.
- All users can read and execute it — any user on the system can run git commands.
- No user can write to it — unprivileged users cannot tamper with the Git binary itself.

Individual Git repositories run as the user who owns the working directory. Git does not require elevated privileges for normal operations. This follows the principle of least privilege: Git only has access to the files it needs to operate on, not the entire system.

This design matters for security. A vulnerability in Git's processing of a repository (such as the path traversal vulnerabilities that have been discovered and patched over the years) cannot automatically escalate to root access, because Git itself does not run as root.

Service Management

Git itself is not a daemon or service — it is a command-line tool that runs, completes, and exits. However, Git-related services (such as git-daemon, which serves repositories over the git:// protocol) can be managed as systemd services:

```
# Start git-daemon as a service
sudo systemctl start git.socket

# Check service status
sudo systemctl status git.socket

# Enable at boot
sudo systemctl enable git.socket

# Stop the service
sudo systemctl stop git.socket
```

For most developers, Git is simply invoked from the command line inside a repository directory. The key commands for day-to-day use are:

```
git init          # Initialize a new repository
git clone <url>   # Clone a remote repository
git add <file>    # Stage changes
git commit -m ''  # Commit staged changes
git push          # Push to remote
git pull          # Fetch and merge from remote
git log           # View commit history
```

Update Model

Security patches and updates to Git reach users through their distribution's package management system. When a vulnerability is discovered and patched in the upstream Git project (hosted at github.com/git/git), the following happens:

- The Git security team reviews and validates the fix privately.
- A new release is tagged in the upstream repository, with a detailed changelog listing the CVE (Common Vulnerabilities and Exposures) identifier.
- Distribution maintainers (Debian, Ubuntu, Fedora, etc.) package the new version and push it to their repositories.
- Users receive the update via their standard system update process (sudo apt upgrade, sudo dnf update, etc.).

This open pipeline means that security vulnerabilities are publicly disclosed, patched, and the fix is available to every user simultaneously — regardless of whether they are running a free community edition or a paid enterprise distribution. There is no paid-only security tier.

Part C — The FOSS Ecosystem

Dependencies

Git is written primarily in C, with some shell scripts and Perl for auxiliary tools. Its compile-time and runtime dependencies are all open-source:

- OpenSSL or LibreSSL — for SHA-1 hashing and HTTPS transport security. Git's cryptographic integrity depends on these.
- libcurl — for HTTP/HTTPS remote operations (git clone, git push, git pull over HTTPS).
- libexpat — for parsing certain XML-based data formats used in some Git extensions.
- zlib — for compression of Git's object store. All stored objects are compressed.
- GNU C Library (glibc) and the GNU Compiler Collection (gcc) — the foundational C runtime and compiler used to build Git itself.
- POSIX shell (bash or dash) — several of Git's helper scripts are shell scripts.

All of these dependencies are themselves open-source projects with their own communities, licenses, and governance structures. Git's existence depends on this broader ecosystem of free software.

What Git Has Enabled

Git's design — distributed, fast, and branching-friendly — has enabled an entirely new generation of software development practices and tools:

- GitHub (2008) — Built entirely on Git's open protocols, GitHub became the largest code-hosting platform in the world. It introduced pull requests, which became the dominant model for code review. GitHub was acquired by Microsoft for $7.5 billion in 2018.
- GitLab (2011) — A fully open-source alternative to GitHub. GitLab's own source code is available under a mixed open-core model. It introduced integrated CI/CD pipelines directly into the Git workflow.
- Gitea and Forgejo — Lightweight, self-hosted Git servers for teams or individuals who want to control their own infrastructure.
- libgit2 — A portable, C-linkable implementation of the Git core logic, allowing Git functionality to be embedded in other applications (used by GitHub Desktop, VS Code's Git integration, and many other tools).
- JGit — A pure Java implementation of Git, used by Eclipse IDE and Gerrit (Google's code review system).
- Infrastructure-as-Code and GitOps — The practice of storing infrastructure configuration in Git and using Git's history and branching model to manage deployments. Tools like Terraform, Kubernetes, and ArgoCD all treat Git as the source of truth for infrastructure state.

Git and the LAMP Stack

Git is not part of the LAMP stack (Linux, Apache, MySQL, PHP) in a technical sense — it is not a web server, database, or runtime. However, it is the foundational tool through which LAMP applications are developed, versioned, and deployed:

- Linux: The OS that runs Git and all LAMP components. The Linux kernel itself is managed with Git.
- Apache: Apache's own source code is managed in a Git repository (git.apache.org). Developers contribute patches to Apache via Git pull requests.
- MySQL: MySQL's source code is on GitHub, managed via Git. MariaDB, the community fork, also uses Git.
- PHP: PHP's source code is hosted at github.com/php/php-src. All PHP contributions go through Git.

In practice, every modern web deployment pipeline uses Git at its centre: code is developed in Git branches, reviewed via pull requests, merged to main, and deployed via CI/CD systems triggered by Git events. Git is to modern web development what the command line is to Linux: the invisible but essential substrate on which everything else runs.

Community and Governance

The Git project is maintained by a distributed community of contributors, with Junio C. Hamano as the principal maintainer since 2005. The governance model is a meritocracy: contributors submit patches to the git@vger.kernel.org mailing list, patches are reviewed by other contributors, and accepted patches are merged by the maintainer.

- Mailing list: git@vger.kernel.org — the primary development discussion list. All patch submissions go here.
- Source repository: github.com/git/git — the canonical Git repository for Git itself.
- Release cadence: approximately quarterly releases, with point releases for critical security fixes.
- Git Merge conference: an annual conference for Git contributors and users.

There is no single corporate entity that controls Git. Microsoft employs several major Git contributors (as part of their investment in developer tools), but the project's direction is governed by its maintainer and community, not by any company's roadmap.

Part D — Open Source vs Proprietary

The closest proprietary alternatives to Git are Perforce (Helix Core), Plastic SCM, and Microsoft's older Team Foundation Version Control (TFVC). Below is a structured comparison across key dimensions:

Dimension	Git (Open Source)	Proprietary Alternatives
Cost	Free (GPL v2). Anyone can download, use, modify, and redistribute Git without cost.	GitHub Desktop: Free tier limited. Perforce, Plastic SCM, SourceSafe: paid per-seat licensing often thousands of dollars annually.
Security	Fully auditable source code. Security researchers worldwide can and do review Git's codebase. CVEs are patched and published openly.	Proprietary tools are black boxes. Users must trust the vendor's security claims without independent verification.
Support & Reliability	Community support via mailing lists, Stack Overflow, GitHub Issues. Extensive documentation. No SLA, but widely deployed with proven stability.	Vendor-provided SLAs, dedicated support teams. Predictable support contracts — important for enterprises with compliance needs.
Freedom to Modify	Full freedom to fork, patch, and redistribute Git. Many distributions (JGit, libgit2) have emerged from this freedom.	No source access. Users are locked into the vendor's feature roadmap and release cycle.
Community vs Corporate Control	Git is governed by its maintainers and the community. Linus Torvalds remains involved. Decisions are made in the open.	Roadmap is determined by profit motive and corporate strategy. Community feedback influences but does not govern decisions.
Ecosystem Integration	Integrates with virtually every IDE, CI/CD system, and development tool. GitHub, GitLab, Bitbucket all built on Git.	Proprietary tools often lock integrations to their own ecosystem, requiring middleware or additional paid connectors.

Overall Verdict

Based on the analysis above, I would recommend Git without hesitation for any real-world software development deployment, from individual projects to large enterprise teams. The technical case for Git is overwhelming: it is faster, more reliable under concurrent use, better supported by the tool ecosystem, and more resilient to infrastructure failures than any proprietary alternative. Its distributed model means that every developer has a complete backup of the entire project history at all times.

The philosophical case is equally strong. Git's GPL v2 license guarantees that no vendor can take away the freedoms the community currently enjoys. There is no risk of a pricing change, a license revocation, or a vendor acquisition that locks teams into proprietary formats. The 2005 BitKeeper incident — the

very event that caused Git to be created — is the best possible argument for why open-source tooling matters for critical development infrastructure.

As for contributing to Git: yes, unambiguously. Contributing to Git means contributing to a piece of infrastructure that underlies virtually all software development. The contribution model (patch submission to the mailing list) is technically demanding, but the reward is participating in the maintenance of a tool used by over 90% of the world's software developers. Even small contributions — documentation improvements, bug reports, test cases — have outsized impact when the software has this many users.

Shell Script Tasks

The following five shell scripts demonstrate practical Linux and shell scripting skills. Each script is documented with its code, an explanation of what it does, and the shell scripting concepts it uses. All scripts are written for bash and tested on Ubuntu 22.04 LTS.

Script 1 — System Identity Report

This script introduces the Linux system like a welcome screen. It displays the distribution name, kernel version, current logged-in user and their home directory, system uptime, current date/time, and a message identifying the open-source license covering the OS.

Concepts demonstrated: Variables, echo, command substitution $(), /etc/os-release parsing for distribution detection, and formatted output.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Abhay Raj] | Reg No: [24BAI10391]
# Course: Open Source Software | Chosen Software: Git

STUDENT_NAME="[Abhay Raj]"
REG_NUMBER="[24BAI10391]"
SOFTWARE_CHOICE="Git"

# Command substitution: execute commands and capture output into variables
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATETIME=$(date '+%A, %d %B %Y — %I:%M %p')

# Read distro name from the standard /etc/os-release file
if [ -f /etc/os-release ]; then
    DISTRO=$(grep -w "PRETTY_NAME" /etc/os-release | cut -d= -f2 | tr -d '"')
else
    DISTRO="Unknown Linux Distribution"
fi

OS_LICENSE="GNU General Public License v2 (GPL v2)"

echo "============================================================"
echo "        OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT         "
echo "============================================================"
echo "  Student   : $STUDENT_NAME ($REG_NUMBER)"
echo "  Software  : $SOFTWARE_CHOICE (under audit)"
echo "------------------------------------------------------------"
echo "  Distribution : $DISTRO"
```

```
echo "  Kernel        : $KERNEL"
echo "  Logged User   : $USER_NAME"
echo "  Home Dir      : $HOME_DIR"
echo "  System Time   : $DATETIME"
echo "  Uptime        : $UPTIME"
echo "------------------------------------------------------------"
echo "  OS License    : $OS_LICENSE"
echo "============================================================"
```

Output:

```
root@ubuntu:~$ nano script1_system_identity.sh
root@ubuntu:~$ chmod +x script1_system_identity.sh
root@ubuntu:~$ ./script1_system_identity.sh
============================================================
        OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT
============================================================

  Student    : [Abhay Raj] ([24BAI10391])
  Software   : Git (under audit)

------------------------------------------------------------
  SYSTEM INFORMATION
------------------------------------------------------------
  Distribution : Ubuntu 24.04.4 LTS
  Kernel       : 6.8.0-101-generic
  Logged User  : root
  Home Dir     : /root
  System Time  : Monday, 30 March 2026 — 07:37 PM
  Uptime       : up 41 minutes

------------------------------------------------------------
  LICENSE INFORMATION
------------------------------------------------------------
  The operating system running this audit is covered under:
  GNU General Public License v2 (GPL v2)

  This means you are free to:
    [1] Run the OS for any purpose
    [2] Study and modify its source code
    [3] Redistribute copies of the OS
    [4] Distribute your modified versions

============================================================
  Open Source: built openly, shared freely, improved by all.
============================================================
```

Script 2 — FOSS Package Inspector

This script checks whether the chosen FOSS package (git) is installed on the system, finds its version and license information using the appropriate package manager, and uses a case statement to print a philosophy note about the package's open-source significance.

Concepts demonstrated: if-then-else, case statement, command availability checking with command -v, dpkg/rpm package queries, pipe with grep.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Checks installation status and prints philosophy notes per package

PACKAGE="git"
INSTALLED=false

echo "============================================================"
echo "          FOSS PACKAGE INSPECTOR"
echo "============================================================"

# Detect package manager and check installation (portable across distros)
if command -v dpkg &>/dev/null; then
    if dpkg -l "$PACKAGE" 2>/dev/null | grep -q "^ii"; then
        INSTALLED=true
        VERSION=$(dpkg -l "$PACKAGE" 2>/dev/null | grep "^ii" | awk '{print $3}')
    fi
elif command -v rpm &>/dev/null; then
    if rpm -q "$PACKAGE" &>/dev/null; then
        INSTALLED=true
        VERSION=$(rpm -qi "$PACKAGE" | grep "^Version" | awk '{print $3}')
    fi
fi

if [ "$INSTALLED" = true ]; then
    echo "  [FOUND] '$PACKAGE' is INSTALLED. Version: $VERSION"
else
    echo "  [MISSING] '$PACKAGE' is NOT installed."
fi

# Case statement: philosophy note based on package name
case $PACKAGE in
    git)    echo "  Git: Born from necessity — Linus built this in a weekend." ;;
    httpd)  echo "  Apache: The server that built the open internet." ;;
    mysql)  echo "  MySQL: Open source at the heart of millions of apps." ;;
    vlc)    echo "  VLC: Students in Paris who wanted to stream freely." ;;
    *)      echo "  '$PACKAGE': Every OSS tool carries a story of sharing." ;;
esac
echo "============================================================"
```

Output:

```
root@ubuntu:~$ chmod +x script2_package_inspector.sh
root@ubuntu:~$ ./script2_package_inspector.sh
==============================================================
          FOSS PACKAGE INSPECTOR
==============================================================

  [FOUND]   'git' is INSTALLED on this system.

  Version   : 1:2.43.0-1ubuntu7.3
  License   : See package documentation
  Summary   : fast, scalable, distributed revision control system

--------------------------------------------------------------
  OPEN SOURCE PHILOSOPHY NOTE
--------------------------------------------------------------
  Git: Born from necessity when Linus Torvalds refused
  to let proprietary tools control Linux development.
  Git is freedom — every clone is a full backup.

==============================================================
```

Script 3 — Disk and Permission Auditor

This script loops through a list of important system directories and reports the permissions, owner, group, and disk usage for each one. It also specifically checks for Git's configuration files and explains their security significance.

Concepts demonstrated: for loop, array iteration, ls -ld, du -sh, awk for field extraction, cut, conditional directory existence checks.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Loops through key directories and reports permissions and disk usage

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/share/doc/git")

echo "============================================================"
echo "          DISK AND PERMISSION AUDITOR"
echo "============================================================"
printf "  %-25s %-8s %-8s %-12s %-10s\n" "Directory" "Owner" "Group" "Permissions"
"Size"
echo "  ----------------------------------------------------------------"

# For loop iterating over the DIRS array
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # awk extracts specific fields from ls -ld output
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')
        # du -sh gives human-readable size; cut -f1 extracts just the value
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "  %-25s %-8s %-8s %-12s %-10s\n" "$DIR" "$OWNER" "$GROUP" "$PERMS"
"${SIZE:-N/A}"
    else
        printf "  %-25s %s\n" "$DIR" "[does not exist]"
    fi
done

# Check Git config files specifically
echo "------------------------------------------------------------"
GIT_GLOBAL_CONFIG="$HOME/.gitconfig"
if [ -f "$GIT_GLOBAL_CONFIG" ]; then
    PERMS=$(ls -l "$GIT_GLOBAL_CONFIG" | awk '{print $1}')
    echo "  [FOUND] ~/.gitconfig | Permissions: $PERMS"
else
    echo "  [INFO] ~/.gitconfig not found — Git not yet configured globally."
fi
echo "============================================================"
```

Output:

```
root@ubuntu:~$ ./script3_disk_auditor.sh
============================================================
          DISK AND PERMISSION AUDITOR
============================================================

  Scanning key system directories...

  Directory                 Owner  Group  Perms       Size
  ----------------------------------------------------------------
  /etc                      root     root     drwxr-xr-x 6.8M
  /var/log                  root     syslog   drwxrwxr-x 20M
  /home                     root     root     drwxr-xr-x 28K
  /usr/bin                  root     root     drwxr-xr-x 501M
  /tmp                      root     root     drwxrwxrwt 80K
  /usr/share/doc/git        root     root     drwxr-xr-x 3.9M

------------------------------------------------------------
  GIT CONFIGURATION DIRECTORY CHECK
------------------------------------------------------------

  Checking Git configuration files...

  [INFO]   ~/.gitconfig not found — Git may not be configured globally yet.
           Run: git config --global user.name 'Your Name'
                git config --global user.email 'you@example.com'

  [INFO]   /etc/gitconfig not found (system-wide config absent — this is normal).

  NOTE: Git's permission model matters for security.
  Config files should be owned by the current user
  and not world-writable (avoid permissions like 777).

============================================================
```

Script 4 — Log File Analyzer

This script accepts a log file and optional keyword as command-line arguments, reads the file line by line using a while-read loop, counts keyword occurrences, implements a do-while-style retry mechanism if the file is not found, and displays the last 5 matching lines.

Concepts demonstrated: while read loop, if-then, counter variables, command-line arguments ($1 and $2), grep, tail, retry loop simulation.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./script4_log_analyzer.sh <logfile> [keyword]

LOGFILE=$1
KEYWORD=${2:-"error"}    # Default keyword: 'error'
COUNT=0
ATTEMPTS=0
MAX_ATTEMPTS=3

# do-while style retry: check file existence, retry up to 3 times
while [ ! -f "$LOGFILE" ] && [ $ATTEMPTS -lt $MAX_ATTEMPTS ]; do
    ATTEMPTS=$((ATTEMPTS + 1))
    echo "  [ATTEMPT $ATTEMPTS/$MAX_ATTEMPTS] Not found: $LOGFILE"
    if [ $ATTEMPTS -lt $MAX_ATTEMPTS ]; then
        read -rp "  Enter a valid log file path (Enter to abort): " NEW_PATH
        [ -z "$NEW_PATH" ] && echo "Aborted." && exit 1
        LOGFILE="$NEW_PATH"
    fi
done

[ ! -f "$LOGFILE" ] && echo "File not found after $MAX_ATTEMPTS attempts." && exit
1
[ ! -s "$LOGFILE" ] && echo "File is empty." && exit 0

echo "  Log File : $LOGFILE | Keyword: '$KEYWORD'"

# while read loop: read one line at a time
while IFS= read -r LINE; do
    # if-then: check each line for the keyword (case-insensitive)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "  RESULT: '$KEYWORD' found $COUNT time(s) in $LOGFILE"
echo "  Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5 | while IFS= read -r L; do
    echo "    >> $L"
done
```

Output:

```
[] Headers:map[]} ImageDecryption:{KeyModel:node} DisableTCPService:true StreamServerAddress:127.0.0.1 Stre
amServerPort:0 StreamIdleTimeout:4h0m0s EnableSelinux:false SelinuxCategoryRange:1024 SandboxImage:registry
.k8s.io/pause:3.5 StatsCollectPeriod:10 SystemdCgroup:false EnableTLSStreaming:false X509KeyPairStreaming:{
TLSCertFile: TLSKeyFile:} MaxContainerLogLineSize:16384 DisableCgroup:false DisableApparmor:false RestrictO
OMScoreAdj:false MaxConcurrentDownloads:3 DisableProcMount:false UnsetSeccompProfile: TolerateMissingHugetl
bController:true DisableHugetlbController:true DeviceOwnershipFromSecurityContext:false IgnoreImageDefinedV
olumes:false NetNSMountsUnderStateDir:false EnableUnprivilegedPorts:false EnableUnprivilegedICMP:false Enab
leCDI:false CDISpecDirs:[/etc/cdi /var/run/cdi] ImagePullProgressTimeout:5m0s DrainExecSyncIOTimeout:0s Ima
gePullWithSyncFs:false IgnoreDeprecationWarnings:[]} ContainerdRootDir:/var/lib/containerd ContainerdEndpoi
nt:/run/containerd/containerd.sock RootDir:/var/lib/containerd/io.containerd.grpc.v1.cri StateDir:/run/cont
ainerd/io.containerd.grpc.v1.cri}"
  >> 2026-03-30T18:34:15.387787+00:00 ubuntu dockerd[951]: time="2026-03-30T18:34:15.387704727Z" level=warn
ing msg="Error (Unable to complete atomic operation, key modified) deleting object [endpoint_count 67aa7bde
b680b246bc065adbd4d8fbb72ac1742d1261c14570d7d55072ff2268], retrying...."
  >> 2026-03-30T18:34:15.517935+00:00 ubuntu dockerd[951]: time="2026-03-30T18:34:15.517773632Z" level=warn
ing msg="CDI setup error /etc/cdi: failed to monitor for changes: no such file or directory"
  >> 2026-03-30T18:34:15.518091+00:00 ubuntu dockerd[951]: time="2026-03-30T18:34:15.517795805Z" level=warn
ing msg="CDI setup error /var/run/cdi: failed to monitor for changes: no such file or directory"

==============================================================
  Tip: Open-source tools like grep, awk, and sed — all
  GPL-licensed — make log analysis powerful and free.
==============================================================
root@ubuntu:~$
root@ubuntu:~$ ./script4_log_analyzer.sh /var/log/syslog error
==============================================================
           LOG FILE ANALYZER
==============================================================

  Log File : /var/log/syslog
  Keyword  : 'error'

--------------------------------------------------------------
  Scanning file line by line...
--------------------------------------------------------------

  RESULT: Keyword 'error' was found 14 time(s) in:
          /var/log/syslog

--------------------------------------------------------------
  LAST 5 MATCHING LINES (most recent occurrences):
--------------------------------------------------------------

  >> 2026-03-30T18:34:14.617223+00:00 ubuntu containerd[628]: time="2026-03-30T18:34:14.611515936Z" level=i
nfo msg="skip loading plugin \"io.containerd.internal.v1.tracing\"..." error="skip plugin: tracing endpoint
 not configured" type=io.containerd.internal.v1
  >> 2026-03-30T18:34:14.617771+00:00 ubuntu containerd[628]: time="2026-03-30T18:34:14.613375634Z" level=i
nfo msg="Start cri plugin with config {PluginConfig:{ContainerdConfig:{Snapshotter:overlayfs DefaultRuntime
Name:runc DefaultRuntime:{Type: Path: Engine: PodAnnotations:[] ContainerAnnotations:[] Root: Options:map[]
 PrivilegedWithoutHostDevices:false PrivilegedWithoutHostDevicesAllDevicesAllowed:false BaseRuntimeSpec: Ne
tworkPluginConfDir: NetworkPluginMaxConfNum:0 Snapshotter: SandboxMode:} UntrustedWorkloadRuntime:{Type: Pa
th: Engine: PodAnnotations:[] ContainerAnnotations:[] Root: Options:map[] PrivilegedWithoutHostDevices:fals
e PrivilegedWithoutHostDevicesAllDevicesAllowed:false BaseRuntimeSpec: NetworkPluginConfDir: NetworkPluginM
axConfNum:0 Snapshotter: SandboxMode:} Runtimes:map[runc:{Type:io.containerd.runc.v2 Path: Engine: PodAnnot
ations:[] ContainerAnnotations:[] Root: Options:map[BinaryName: CriuImagePath: CriuPath: CriuWorkPath: IoGi
d:0 IoUid:0 NoNewKeyring:false NoPivotRoot:false Root: ShimCgroup: SystemdCgroup:true] PrivilegedWithoutHos
tDevices:false PrivilegedWithoutHostDevicesAllDevicesAllowed:false BaseRuntimeSpec: NetworkPluginConfDir: N
etworkPluginMaxConfNum:0 Snapshotter: SandboxMode:podsandbox}] NoPivot:false DisableSnapshotAnnotations:tru
e DiscardUnpackedLayers:false IgnoreBlockIONotEnabledErrors:false IgnoreRdtNotEnabledErrors:false} CniConfi
g:{NetworkPluginBinDir:/opt/cni/bin NetworkPluginConfDir:/etc/cni/net.d NetworkPluginMaxConfNum:1 NetworkPl
uginSetupSerially:false NetworkPluginConfTemplate: IPPreference:} Registry:{ConfigPath: Mirrors:map[docker.
io:{Endpoints:[https://mirror.gcr.io https://docker-mirror.killercoda.com https://docker-mirror.killer.sh h
ttps://registry-1.docker.io]}] Configs:map[registry.killer.sh:5000:{Auth:<nil> TLS:0xc0006ee000}] Auths:map
```

Script 5 — The Open Source Manifesto Generator

This script interactively asks the user three questions, then composes a personalised open-source philosophy statement using their answers. The result is saved to a .txt file and printed to the terminal.

Concepts demonstrated: read for interactive user input, string concatenation by embedding variables in double-quoted strings, file redirection (> and >>), the date command, and the alias concept demonstrated via a comment.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Generates a personalised open-source philosophy statement

# Alias concept: in a real shell session you might write:
#   alias today='date "+%d %B %Y"'
# Here we use a variable for the same readable result:
TODAY=$(date '+%d %B %Y')
OUTPUT_FILE="manifesto_$(whoami).txt"

echo "Answer three questions to generate your manifesto."
echo ""

# read: interactive input — -p shows the prompt inline
read -rp "1. Name one open-source tool you use every day: " TOOL
read -rp "2. In one word, what does 'freedom' mean to you in tech? " FREEDOM
read -rp "3. Name one thing you would build and share freely: " BUILD

# Write manifesto to file using > (create) and >> (append)
echo "============================================" > "$OUTPUT_FILE"
echo "       MY OPEN SOURCE MANIFESTO" >> "$OUTPUT_FILE"
echo "   Generated: $TODAY by $(whoami)" >> "$OUTPUT_FILE"
echo "============================================" >> "$OUTPUT_FILE"
echo "" >> "$OUTPUT_FILE"

# String concatenation: variables embedded in double-quoted strings
echo "Every day, I rely on $TOOL — a tool built not for profit," >> "$OUTPUT_FILE"
echo "but for people. Its existence reminds me that $FREEDOM" >> "$OUTPUT_FILE"
echo "matters more than lock-in, patents, or paywalls." >> "$OUTPUT_FILE"
echo "" >> "$OUTPUT_FILE"
echo "If I were to build $BUILD, I would share it freely." >> "$OUTPUT_FILE"
echo "Open source is not a license — it is a philosophy." >> "$OUTPUT_FILE"
echo "" >> "$OUTPUT_FILE"
echo "  — $(whoami) | $TODAY" >> "$OUTPUT_FILE"

echo "Manifesto saved to $OUTPUT_FILE"
cat "$OUTPUT_FILE"
```

Ouput:

```
Terminal 0 ×

root@ubuntu:~$ nano script5_manifesto_generator.sh
root@ubuntu:~$ chmod +x script5_manifesto_generator.sh
root@ubuntu:~$ ./script5_manifesto_generator.sh
============================================================
       OPEN SOURCE MANIFESTO GENERATOR
============================================================

  This tool will create a personalised open-source
  philosophy statement based on your answers.

  Answer three short questions below.

------------------------------------------------------------
  1. Name one open-source tool you use every day: git

  2. In one word, what does 'freedom' mean to you in tech? choice

  3. Name one thing you would build and share freely: music app

------------------------------------------------------------
  Generating your manifesto...
------------------------------------------------------------

  [SUCCESS] Manifesto saved to: manifesto_root.txt

------------------------------------------------------------
  YOUR MANIFESTO:
------------------------------------------------------------

============================================================
           MY OPEN SOURCE MANIFESTO
    Generated on 30 March 2026 by root
============================================================

Every day, I rely on git — a tool built not for profit,
but for people. Its existence reminds me that some of the
most powerful things in technology were given away freely,
by people who believed that choice matters more than
lock-in, patents, or paywalls.
```

```
I believe in the open-source way. If I were to build
music app, I would share it. Not because I have to, but
because the tools I use every day exist because someone
before me made that same choice. Standing on the shoulders
of giants means carrying the ladder forward — releasing
your work so the next person climbs higher than you did.

Open source is not a license model. It is a philosophy:
that knowledge shared is knowledge multiplied, and that
the best software is built when no one owns the problem.

  — root | 30 March 2026

============================================================
root@ubuntu:~$
```

Conclusion

This audit of Git has covered far more ground than the history of a version control system. It has traced the moment when proprietary software failed an open community, and the community's response was not to negotiate better terms with the vendor — it was to build something freer and better and give it to the world. That story repeats throughout the history of open-source software, and it is the reason why the foundational tools of modern computing — the kernel, the compiler, the web server, the database, the version control system — are overwhelmingly open.

The four freedoms encoded in the GPL v2 are not abstract philosophy. They are the structural guarantees that prevented Git from ever being enclosed by a company, that ensure security vulnerabilities are publicly disclosed and patched, and that allow anyone with the ability to read C code to verify that the tool behaves as described. These freedoms have real, measurable consequences for the security, reliability, and longevity of the software ecosystem.

The ethical questions explored in Part A do not have easy answers, but they are questions every software professional will encounter in some form throughout their career: who owns the code you build? Who benefits when you give it away? What obligations do you carry to the community of people whose work you build on? Git's story — from the BitKeeper dispute to the world's most widely deployed development tool — is one of the clearest object lessons in how those questions play out in practice.

Writing the five shell scripts for this project was a reminder that the tools explored in this report — bash, grep, awk, the entire GNU toolchain — are themselves open-source software with their own origin stories, their own license texts, and their own communities. Every command typed in a Linux terminal is a small act of standing on the shoulders of giants who chose to share their work freely. Understanding that is, as the course materials say, not a footnote to a technical education. It is the foundation of one.

References

GNU Project — The Free Software Definition: gnu.org/philosophy/free-sw.html

GPL v2 Full License Text: gnu.org/licenses/old-licenses/gpl-2.0.html

Open Source Initiative — The Open Source Definition: opensource.org/osd

Eric S. Raymond — The Cathedral and the Bazaar: catb.org/~esr/writings/cathedral-bazaar

Linus Torvalds — Git initial commit message and release announcement (April 2005), available in the Git repository history: github.com/git/git

The Linux Command Line by William Shotts — linuxcommand.org

GNU Bash Manual: gnu.org/software/bash/manual

SPDX License List — spdx.org/licenses

Git official website and documentation: git-scm.com

GitHub Blog — The story of Git: github.blog